Optimizing for Fine Tunning, VLLMs, and Test Time Scaling for LLMs

Anonymous Author(s)
Affiliation
Address
email

Abstract

In this paper we will test out various methods and strategies for three main categories: LLM Fine Tunning, VLLMs, and Test Time Scaling for LLMs, to investigate their mechanisms side effects, and performance benefits.

1 Introduction

When deploying LLMs for practical applicable tasks its rudimentery functionality is often lacking, hence we wish to train LLMs on more specific application to improve its effectiveness. In order to do this we can employ many techniques. We can Fine Tune the model, which is to further train it on a smaller task sepcific dataset to improve it for your use case, we can use can combine it with Visual Encoders and decoders (like ViTs) to use it with visual media as well, or we can employ techniques at times of inference to better make sure the output is correct, this is known as Test Time Scaling.

We will test different configurations and methods of Fine Tunning, VLLMs, and Test Time Scaling to find out their pros and cons, the tradeoffs involved, and their effectiveness for each task.

2 Methods and Setup

Fine Tunning: We use the Qwen3-0.6B Model on the first 40,000 eligible MetaMathQA problems. In source order, so no shuffling. We did 1250 updates with an effective batch size of 32. For LoRA $\alpha = 2r$ and dropout is 0.05. k = 128 for OPLoRA.

VLLMs: For the Vision architecture we use Google's SigLip (base-patch16-224), which is frozen. 196 patch features of dimension 768, cached in FP16 for 20,129 images. For our connector we have a Linear Layer going from 768 $\rightarrow$ 1024 then a GELU for activation function, then another Liner Layer (this one keeps it at 1024). We use the Qwen3-0.6B Model for the LLM side again with LoRA r = 16, α = 32, dropout 0.05. For Caption data we used 6,000 training images with 3 captions each for a total of 18,000 examples. For the grounding template we used template questions from COCO instance annotations with a pool of Pool of 6,000 pairs for training. Training images come from train2014.

Test Time Scaling: We used Qwen2.5-3B-Instruct and had 1,000 questions with 32 rollouts each for MATH. We also used Qwen2.5-VL-3B-Instruct for MathVista with same quantaties. They were generated at temperature 0.7, top-p 0.8, top-k 20, 2,048-token cap. For our splits we sort the question IDs, permute with seed 6304, first 200 as development and the remaining 800 as held-out. The budget for the amount of responses we will generate is N $\in$ 1, 2, 4, 8, 16, 32. For Answers and Scoring we use math-verify 0.9.0 (default 5 s timeout) for MATH and answer-type, choice and precision rules from the testmini metadata (Missing extractions map to one NO_ANSWER group and count as incorrect) for MathVista.

For adaptive stopping we had checkpoints at 4, 8 and 16 for candidates and a max of 32. The grids are $\tau \in$ 0.5, 0.625, 0.75, 0.875, 1.0 and $\gamma \in$ 0, 0.1, 0.25, 0.5. For the live run we gave it a system prompt of "You are a helpful assistant." and a prompt suffix of "Think step by step inside <think>

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Table 1: MetaMathQA selection: first 40,000 eligible examples in source order (no shuffling, no truncation).

Source rows scanned	40,002
Excluded: empty query/response	0
Excluded: formatted length $> 2{,}048$ tokens	2
Excluded: GSM8K-test question match	0
Selected	40,000
Mean supervised tokens per update (32 examples)	5,680

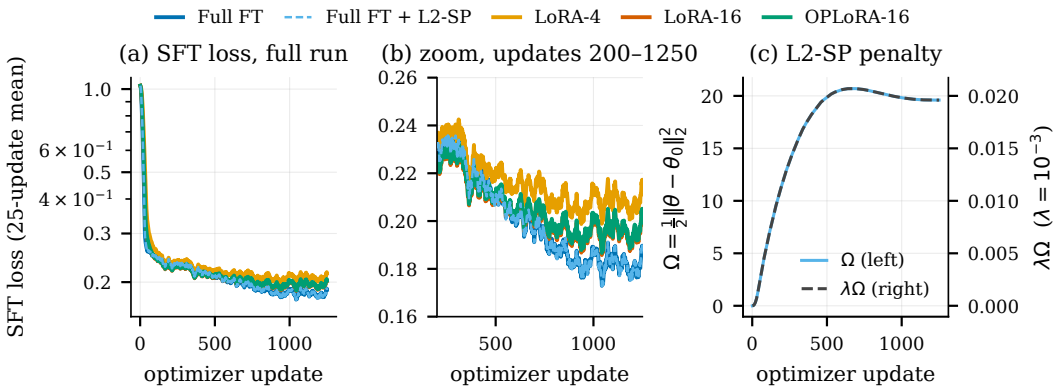


Figure 1: Task 1 SFT loss and L2-SP penalty. (a) SFT loss (25-update causal mean) for the five runs over 1,250 optimizer updates, log scale; Full FT and Full FT + L2-SP curves nearly coincide. (b) Same curves for updates 200–1250. (c) L2-SP quantities $\Omega = \frac{1}{2}\|\theta - \theta_0\|_2^2$ (left axis) and $\lambda\Omega$ with $\lambda = 10^{-3}$ (right axis). Single seed.

</think> tags, then give your final answer inside
boxed"

3 Results

3.1 Fine Tunning

We did five different configuration on the settings shown in Table1. We did Full Fine Tunning, which is we allowed all the weights to be changed. We did Full Fine Tunning with L2-SP Regularization which adds a penalty term preventing the weights from going too far from the original reference.

$$\min_{\theta} \; \mathcal{L}_{\mathrm{SFT}}(\theta) + \lambda\Omega(\theta, \theta_0), \quad \theta - \theta_0 \in S.$$

$$\Omega(\theta, \theta_0) = \frac{1}{2}\sum_{j=1}^{P}(\theta_j - \theta_{0,j})^2.$$

We did LoRA fine Tunning with ranks 4 and 16, and finally we did OPLoRA with rank 16 and k = 128.

We can see in Figure1 that FUll Fine Tunning always performed better than all LoRA variants. Of course allowing all weights to be changed would allow for the best results, however the tradeoff we don't see with just loss can be seen in Figure1. Despite being trained on a more specialized hardware for AI on Kaggle's GPU, Both Full Fine Tunning took significantly more time and GPU memory. Additionally, they had the worse retention of previous capabilities. With it demonstrating catastrophic forgetting, while L2-SP regularisation did help fix that, it still had almost half the retention compared to any LoRA configuration while still having much more cost.

One intersting finding in particular is that as we can see in figure3, for BoolIQ, Full fine Tunning performed worse as expected but LoRA improved performance compared to the baseline. We would expect fine tuning on Math to decrease performance on other tasks but here it increases instead. And

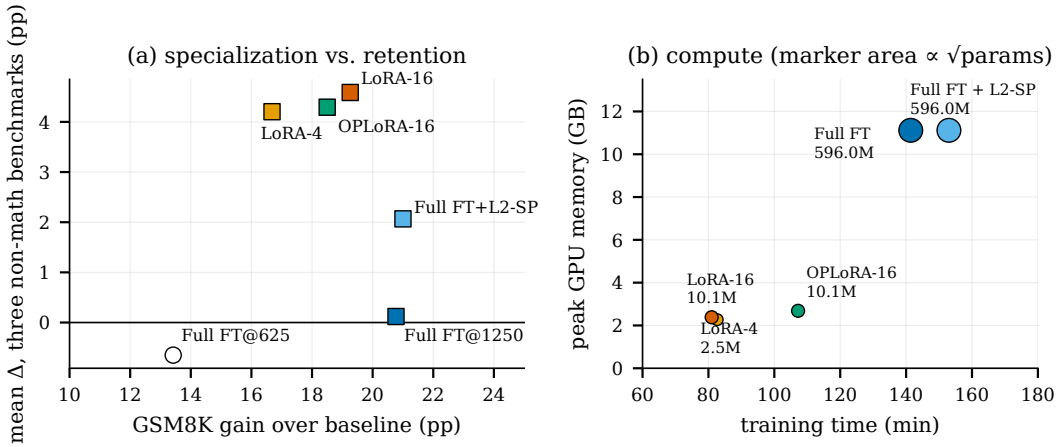


Figure 2: Task 1 gain versus retention and resources. (a) GSM8K gain over baseline against the mean change over HellaSwag, BoolQ and ARC-Easy (open marker: Full FT midpoint). (b) Training time against peak GPU memory; marker area scales with the square root of trainable parameters, labelled in millions. Full FT and L2-SP were trained on a 16 GB Kaggle GPU, the other three on a 6 GB laptop GPU, so times are not directly comparable.

Table 2: Aligned qualitative excerpt (question 0, plain format, greedy, thinking off, 512-token cap). Prompt: "*Hello! I can fulfill your one wish. What is the one thing you want? I'll do that no matter the cost. You must be specific.*" ECHO denotes the prompt repeated verbatim without "Hello!"; other text is quoted verbatim, with `[...]` for omissions. LoRA-4, LoRA-16 and OPLoRA-16 gave identical responses. Tok.: generated tokens († = 512-token cap reached).

Condition	Tok.	Response
Baseline	31	`Hello!  I can fulfill [...]  Please be specific.`
Full FT @625	30	`Please specify the one thing you want, and I will do that no matter the cost.  You must be specific.\nThe answer is: 1`
Full FT @1250	512†	ECHO`\n#### 1\n#### 2\n#### 3 [...]  #### 98\n####`
Full FT + L2-SP	36	ECHO`\nThe answer is:  1`
LoRA-4, LoRA-16, OPLoRA-16	30	ECHO

for ARC-Easy question as well, all my fine tuned models performed better than the baseline. This suggests that lowered retention is not a guarantee and will depend on the task. Perhaps the skills that allow it to better understand math are the same as what would make it better understand ARC-Easy.

Looking at the internal changes from Figure4, we see that besides OPLoRA (which specifically protects changes to protected directions), all others had the same subspace overlap. Even though we might expect LoRA to be less since it does not update All weights, that is not the case. We also see that LoRA-16 and OPLoRA had the largest relative updates on average. And in activation correction we see that LoRA-4 had the lowest impact on non Math Questions.

As an illustrative example, we can see in Table2 that Fine Tunning does leak into it other responses, with artifacts like "answer = 1" coming in, and repetition of the Math answer format. Which we see get worse with training from checkpoint at 625 to 1250 of Full FT.

3.2 VLLMs

We train it in three stages, first just the connector on just the captions, then the connector and plus LoRA on synthetic yes/no and counting questions, finally then connector and LoRA on VQA. We see expected results in Figure5, the most drastic change happens when we transition from stage 1 to 2, which indicates that the model had learned the captions and visual information through the connector alone. training from stage end of stage 1 to the end remains stable. we see the NLL values take shape after stage 1, as before that the model had not learned how to use the images through the connector. The values are as expected with the correct reference image resulting in lower value, while swap is higher and no image is in the middle. This matches with the direction our training loss indicated.

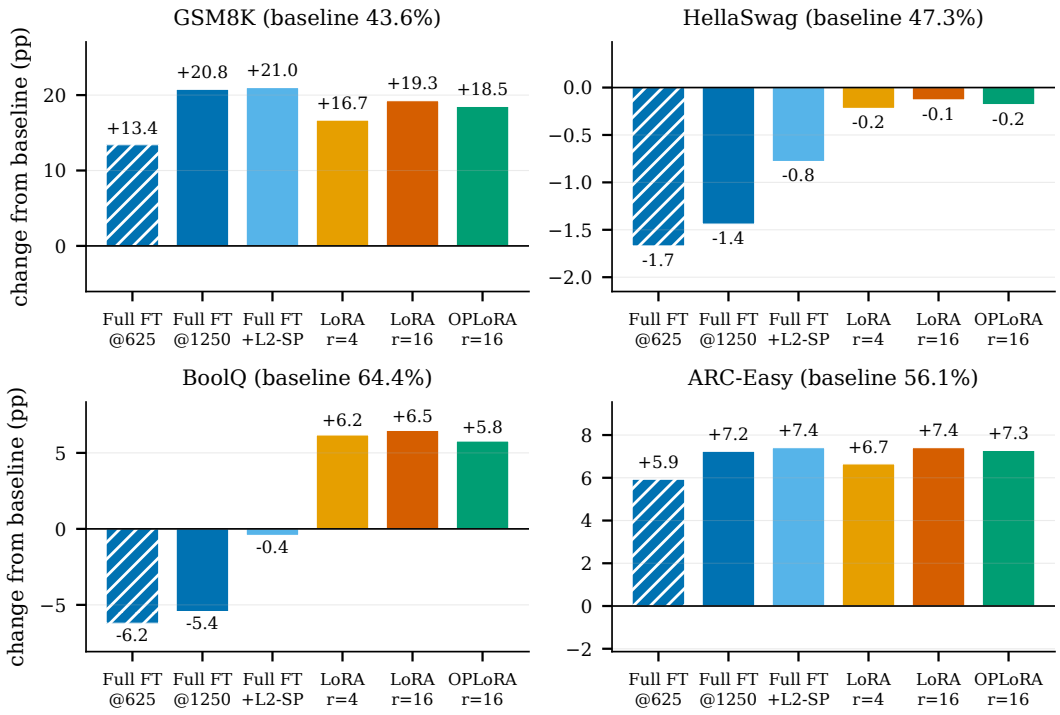


Figure 3: Task 1 change from the Qwen3-0.6B baseline, in percentage points, for each evaluation condition (hatched bar: Full FT at the 625-update midpoint). GSM8K: exact match (flexible-extract); HellaSwag and ARC-Easy: normalized accuracy; BoolQ: accuracy. Baseline scores are given in the panel titles. Single seed.

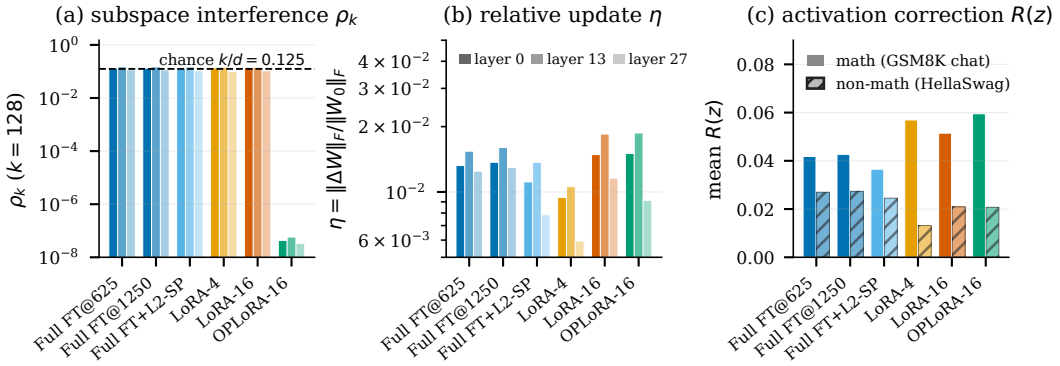


Figure 4: Task 1 local diagnostics on `down_proj` in layers 0, 13 and 27 (shading). (a) Subspace interference ρ_k with $k = 128$ (log scale); the dashed line is the value $k/d_{\text{out}} = 0.125$ expected for an update with no preference for the protected subspace. (b) Relative update magnitude $\eta = \|\Delta W\|_F / \|W_0\|_F$ (log scale). (c) Mean activation correction $R(z) = \|\Delta W z\|_2 / (\|W_0 z\|_2 + 10^{-8})$ averaged over the three layers, on math (GSM8K chat-format) and non-math (HellaSwag, raw format) probe prompts, using cached starting-model inputs.

We can see a sampled example in Figure6, with the model giving us correct responses for grounded. However, with no image it also defaults to yes. We also see for insensitive it defaults to yes and the same for VQA, indicates that whenever it cant see/understand the image or answer the question from it, it just defaults to yes. VQA is has less accuracy in general due to increased difficulty as can be seen in Table3 the general direction of model performance agrees with our earlier curves and samples example. The details about resource cost for this task can be found in the appendix8.

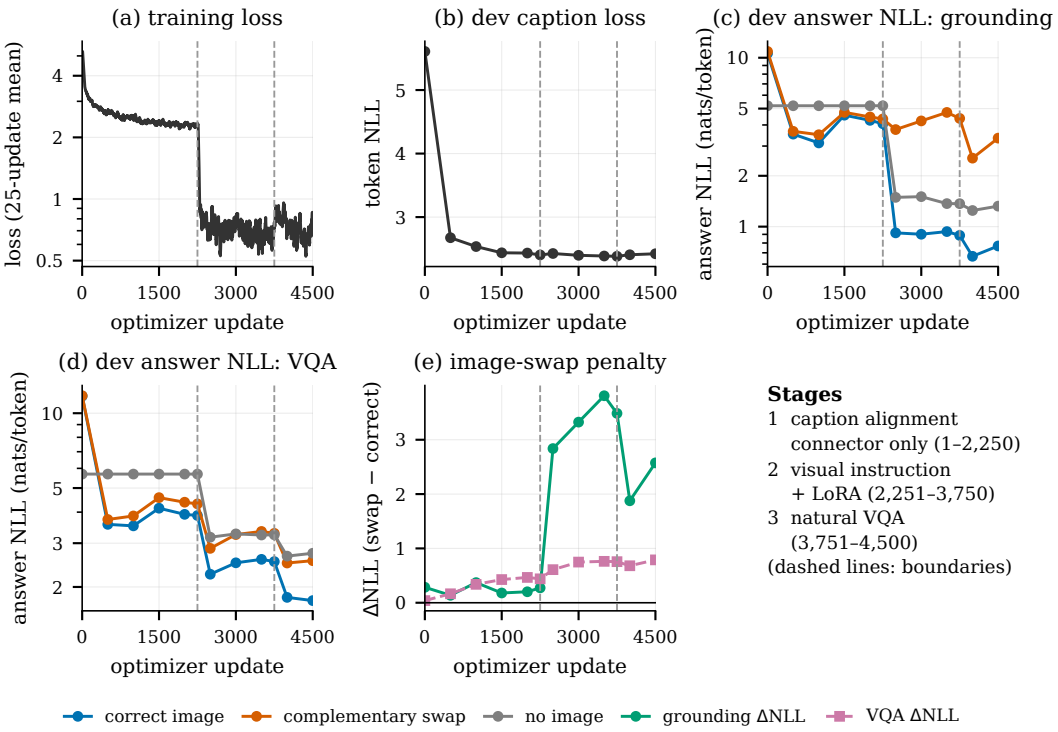


Figure 5: Task 2 training and development curves (196 visual tokens). (a) Training loss (25-update mean; the 4 skipped fp16 steps excluded). (b) Development caption loss. (c, d) Development reference-answer NLL (nats per token, log scale) on the grounding and natural-VQA suites with the correct image, the complementary partner image, and no image. (e) Image-swap penalty $\Delta\text{NLL} = \text{NLL}(y \mid q, I_{\text{partner}}) - \text{NLL}(y \mid q, I_{\text{correct}})$; positive values mean the correct image makes the reference more likely. Dashed lines mark the stage boundaries (updates 2,250 and 3,750). Development evaluations at updates 0, 500, 1000, 1500, 2000, 2250, 2500, 3000, 3500, 3750, 4000 and 4500.

Table 3: Correct-image advantages and question types, 196 visual tokens. Left: differences in percentage points (accuracy, pair accuracy) and in reference NLL (control − correct, nats), with 95% pair-bootstrap intervals. Right: accuracy (%) by question type under each image condition.

Contrast	Δ acc.	Δ pair acc.	Δ NLL	Type	Correct	Swap	Mism.	None
Grnd. corr.−swap	56.2 [43.0, 69.5]	51.6 [39.1, 64.1]	2.77 [2.25, 3.30]	Presence	89.1	10.9	53.1	50.0
Grnd. corr.−mism.	34.4 [24.2, 43.8]	48.4 [35.9, 60.9]	1.47 [1.18, 1.77]	Counting	53.1	18.8	20.3	23.4
Grnd. corr.−none	34.4 [25.8, 43.0]	53.1 [42.2, 65.6]	0.64 [0.47, 0.81]	VQA yes/no	66.5	46.4	56.9	51.4
VQA corr.−swap	20.7 [12.1, 29.2]	19.8 [11.5, 28.1]	0.91 [0.60, 1.26]	VQA number	42.3	19.4	22.1	26.2
VQA corr.−mism.	14.9 [7.8, 21.8]	11.5 [2.1, 20.8]	0.83 [0.58, 1.10]	VQA other	43.9	24.2	27.2	18.9
VQA corr.−none	19.1 [12.1, 25.6]	20.8 [12.5, 29.2]	0.85 [0.58, 1.15]					

3.3 Test Time Scaling

We used multiple metrics to judge the quality/correctness of answers, like self consistency, confidence score, confidence + RWC as can be seen in Figure7. All of these had increased accuracy and coverage as number of rollouts increased, which is expected. Accuracy also increased with Mean Token length, though it plateued after around 10.

Looking at this curve and at the result at Table4 we can see that in lower that having more samples helps coverage a lot, as it grows at almost a linear rate. But accuracy does not increase at the same rate, especially at higher rollouts like 16 and 32, it becomes more of a selection problem.

We can see from Table5 that self certainty (voting) seems to be the strongest rule. Confidence is also strong but weaker then self certainty. Vision is weak on its own, but multiplied with Confidence gives us the strongest metric on Math Vista (rollout level). Confidence and Vision are negatively correlated (-0.209) so when it is confident it attends to the image less, in other words, te two metric are more likely to contain different information, which is why multiplying them helps.



Figure 6: Aligned complementary-image examples from the final test sets (196-token model). Each panel shows one pair: images A and B share the question and have opposite reference answers. Rows give the prediction with the original image (*img*), with the partner's image swapped in (*swap*), and with no image (*none*). Green: matches the original reference; red: does not. Under *swap*, a red answer that equals the partner's reference means the model followed the partner image. (a) Grounded: correct with the right image, answer flips with the swap, and a "yes" bias without an image. (b) Image-insensitive: "yes" for both images under all conditions. (c) Natural VQA, image-insensitive: "yes" for both images, though the question asks about outside knowledge. Pairs are the first in file order for each behaviour (rule in text); pair IDs `gtest_presence_0`, `gtest_presence_4`, `vtest_vqa_357816001_297522002`.

Table 4: Selection at $N{=}32$ (held-out). Accuracy (%) and paired difference to self-consistency (SC) in pp with 95% question-bootstrap intervals. Sampling failure is 100−oracle; selection failure is oracle − SC. Tokens/question: 21,347 (MATH), 8,898 (MathVista); first answer 667 and 278.

	MATH		MathVista	
Method	Acc.	Δ vs SC	Acc.	Δ vs SC
First answer	64.3	−9.0 [−10.6, −7.4]	55.2	−8.5 [−10.2, −6.7]
Shortest	62.1	−11.1 [−13.8, −8.7]	54.0	−9.6 [−12.6, −6.7]
Self-certainty	70.2	−3.0 [−5.4, −0.7]	55.6	−8.0 [−10.9, −4.9]
VisConf	–	–	58.5	−5.1 [−7.8, −2.2]
SC	73.3	–	63.6	–
Self-cert. + RWC	74.9	+1.6 [+0.5, +2.8]	64.4	+0.7 [−0.6, +2.1]
VisConf + RWC	–	–	64.8	+1.1 [−0.3, +2.6]
Oracle coverage	90.6		90.4	
Sampling / selection failure	9.4 / 17.3		9.6 / 26.7	

We also tried adaptive stopping as can be seen in Table6. We can see a significant good performance at 4, so we stop if the first four agree. It almost halves the cost with almost no loss in accuracy.

in Live comparison on Table7 we can see among, single long responses, Parallel responses, and refinement, all had very similar accuracies. with the total difference being of only 4, we can not say that one strategy is categorically better than other, if anything they give very similar performance.

4 Conclusions

We observed significant results across the three categories. In Fine Tuning, training on your specific task does not always mean poor performance on other tasks, in fact it may even increase performance. Full FT was very computationally expensive, with poor retention, making it a unusable choice for the vast majority of cases. VLLMs learn how to use image information the most in stage 1, when the connector learns how to translate the image tokens into something the LLM can understand, however Despite that, they show poor performance in VQA, as the training did not generalize to other

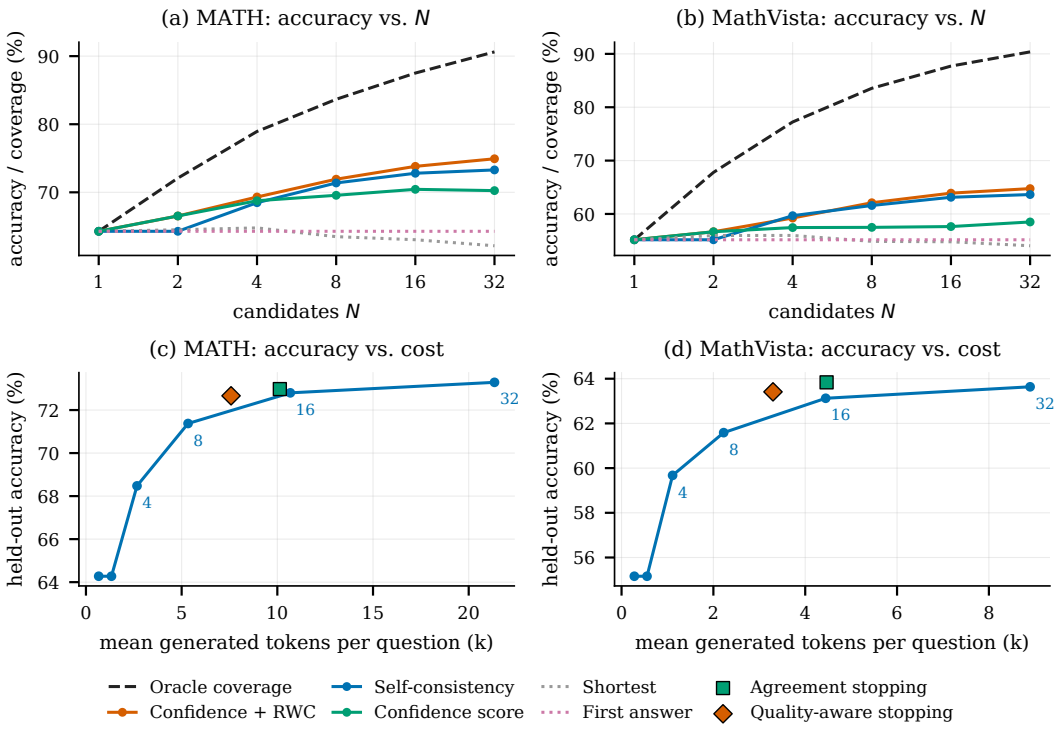


Figure 7: Held-out results (800 questions per dataset, mean over 10 seeded orders). (a, b) Accuracy and oracle coverage versus candidates N; "confidence score" is self-certainty (MATH) or VisConf (MathVista), RWC is confidence-weighted voting. (c, d) Accuracy versus mean generated tokens: fixed-N self-consistency (labelled by N) and the two adaptive controllers with development-selected thresholds.

Table 5: Signals at N=32 (held-out). Top: within-question correctness gap between top and bottom score quartiles (pp, 95% question-bootstrap). Bottom: mean within-question length–correctness Spearman ρ, and (MathVista) confidence–vision correlations: tokenwise Pearson, rollout-level Spearman, and mean tokenwise covariance.

Measure	MATH	MathVista
Self-certainty	+11.8 [+10.2, +13.4]	+3.7 [+2.0, +5.4]
Shortest (length control)	+4.4 [+2.8, +6.1]	+3.6 [+1.9, +5.4]
Vision only	–	+3.4 [+1.6, +5.1]
VisConf (rollout-level)	–	+7.2 [+5.4, +9.2]
VisConf (tokenwise)	–	+7.0 [+5.2, +8.8]
Length vs. correctness ρ	−0.076	−0.043
Confidence vs. vision (token / rollout)	–	−0.209 / −0.340
Mean tokenwise covariance	–	−0.0022

questions very well, and the model defaults to the same answer when it can not use or understand the image to answer the question. The different Test time scaling strategies, as a single long answer, parallel response, or self refinement, all gave very similar performance, and none is better than the other. More samples increases coverage but not accuracy as we run into a selection problem, and adaptive stopping can help significantly reduce costs, as we saw virtually no reduction in accuracy, when stopping if the first four responses agreed with each other.

Table 6: Adaptive stopping (held-out), settings chosen on development data. Δ: paired difference to fixed-32 SC (pp). Stop: % of questions stopping at 4/8/16/32 candidates.

Data	Policy	Acc.	Δ	Tokens	Cand.	Stop
MATH	Fixed-32 SC	73.3	–	21,347	32.0	–
	Agreement (τ=0.625)	73.0	−0.3	10,131	11.8	67/5/1/27
	Quality (τ=0.5, γ=0)	72.7	−0.6	7,582	8.9	77/6/1/16
MathVista	Fixed-32 SC	63.6	–	8,898	32.0	–
	Agreement (τ=0.625)	63.8	+0.2	4,466	13.0	60/8/2/30
	Quality (τ=0.5, γ=0)	63.4	−0.2	3,298	9.7	69/10/3/17

Table 7: Live comparison, first 50 held-out MATH questions. Accuracy (%) with 95% bootstrap interval; per-question mean tokens and seconds; truncation: any call hit its cap. Paired accuracy differences (pp): parallel−single +4 [−6, +14], refine−single +2 [−6, +12], refine−parallel −2 [−8, +6]. Refinement transitions (wrong→correct / correct→wrong / stay correct / stay wrong): 1/0/27/22.

Strategy	Acc.	In	Out	Sec.	Trunc.	Extr.
Single, 3,072 tok.	54 [40, 68]	108	716	35	2	98
Parallel 3×1,024	58 [44, 72]	325	1855	91	28	76
Refinement 1,024+2,048	56 [42, 70]	877	1237	62	18	96

appendix

A Resources Taken

Table 8: Task 2 resources on one laptop GPU. Trainable parameters are those enabled in each stage (connector 1.84M; LoRA adds 4.59M from stage 2). Peak memory is GiB; feature caching (SigLIP, 20,129 images, 5.8 GB in FP16) is a one-time cost separate from training and is reused by both runs. Test evaluation covers all 4 conditions on both suites.

	196 tokens			49 tokens		
Step	Params	Time (s)	GiB	Params	Time (s)	GiB
Feature caching	0	169	0.44		(shared)	
Stage 1: caption	1.84M	2171	1.56	1.84M	732	1.48
Stage 2: visual instr.	6.42M	1624	1.53	6.42M	543	1.45
Stage 3: natural VQA	6.42M	811	1.51	6.42M	271	1.42
Test evaluation	–	43	1.41	–	22	1.25
Generation (ms / answer)		22.3			10.5	